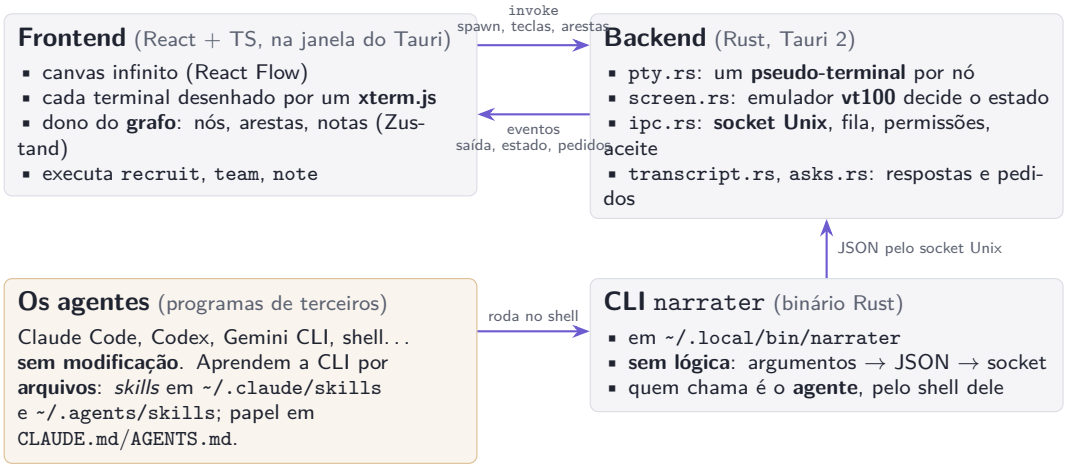
Como o NarraTer é feito

Arquitetura: três peças



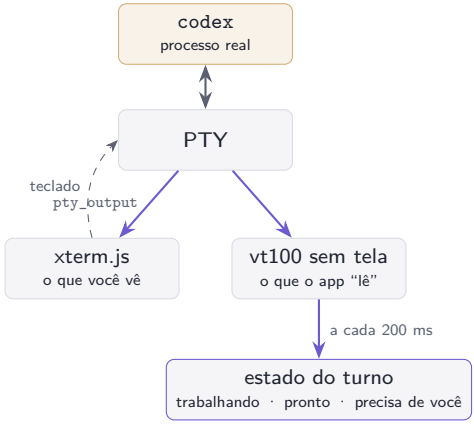
O que é um terminal no NarraTer

Quando um nó de terminal aparece no canvas, o frontend chama `pty_spawn` e o backend:

1. escreve o **papel** em `.narrater/roles/<id>/CLAUDE.md` e `AGENTS.md`, e inicia o agente nessa pasta, porque toda CLI lê esses arquivos sozinha;
2. abre um **pseudo-terminal** (PTY) e roda o programa nele, dentro de uma sessão `tmux` própria quando a sessão é persistente (sobrevive ao app fechar);
3. injeta a **identidade** do terminal no ambiente:

```
NARRATER_SOCKET=/tmp/narrater-<pid>/narrater.sock
NARRATER_ID=<id do nó>   NARRATER_LABEL=Ana
NARRATER_TOKEN=<segredo deste terminal>
```

4. lê a saída do PTY e manda **para dois lugares**.



O agente nunca sabe que está no NarraTer: para ele, é um terminal comum.

O que acontece num `narrater ask`

O agente líder roda, no shell dele:

```
narrater ask "Ana" "Rode os testes e me diga o resultado"
```

A CLI só lê o ambiente e manda ao socket:

```
{"from": "<NARRATER_ID>", "token": "<NARRATER_TOKEN>",
 "mode": "ask", "to": "Ana", "msg": "Rode os testes..."}
```

O socket fica numa pasta só do usuário (permissões 0700/0600). O *token* prova que o pedido veio mesmo daquele terminal: o nome que a CLI manda nunca é aceito sem conferência.

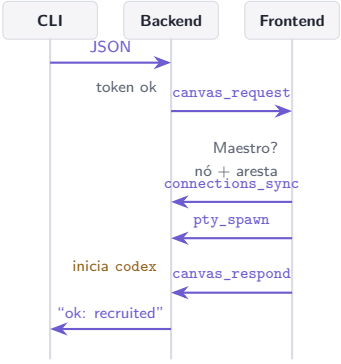
Com `-check "CMD"` (D24): depois do passo 6, o **backend** roda o critério de aceite no projeto do alvo. Se falhar, o pedido termina como `failed`, com a saída do comando: "pronto" passa a ser verificado pelo ambiente, não pela palavra do agente.

1. **Autentica**: o *token* bate com o do terminal `from`?
2. **Autoriza**: existe aresta entre o chamador e "Ana"? (`resolve_target`). Sem aresta, erro.
3. **Registra** o pedido em `asks.jsonl` (sobrevive a reinício) e **anota o tamanho** do *transcript* da Ana, para achar a resposta depois.
4. **Enfileira** `[narrater from Narrador]: Rode os testes...`
5. **Entrega** quando a Ana fica ociosa: *digita* o texto no PTY dela (colagem entre colchetes se o programa aceitar, senão texto + Enter após 300 ms).
6. **Espera o fim do turno**: tela (mudou? rodapé "esc to interrupt"? título da janela?) e *transcript* (`system/turn_duration`).
7. **Responde**: o último texto da Ana no *transcript*, ou a tela se não houver. A CLI imprime, e o líder lê como a saída de qualquer comando.

O que acontece num `narrater recruit`

```
narrater recruit "Ana" --preset Codex --role Developer
```

1. A **CLI** monta o JSON `{mode: canvas, action: recruit, name, preset, role}` e manda ao socket.
2. O **backend** autentica o *token*. Como o grafo pertence ao frontend, ele emite `canvas_request` e espera a resposta.
3. O **frontend** (`managerCommands.ts`) confere se o chamador está em **modo Maestro** (senão, erro), resolve o *preset* (o comando do Codex) e o papel, e cria o **nó** num lugar livre ao lado do líder, com a **aresta** líder → Ana.
4. A mudança no grafo dispara duas coisas: `connections_sync` (o backend recebe a nova tabela de permissões) e `pty_spawn` (o terminal da Ana nasce, como no slide anterior).
5. O frontend devolve `ok: recruited 'Ana' ..., connected to you`, que volta pela CLI ao líder.



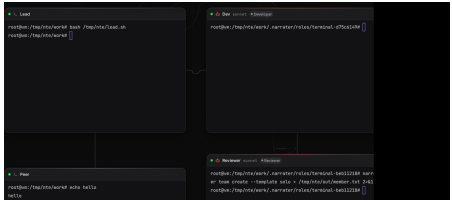
Em resumo: recrutar não cria um "agente" abstrato. Cria um nó, abre um processo real do Codex num pseudo-terminal, entrega as instruções por arquivo e uma aresta que dá permissão de conversar.

O que acontece num `narrater team create`

Monta a equipe inteira numa chamada, a partir de um **formato conhecido**:

```
narrater team create --template dev-reviewer --task "..." \
  --reason "revisor com contexto limpo" --model sonnet
```

1. Mesmo caminho do `recruit`: CLI → socket → backend → `canvas_request` → frontend (`teamCommands.ts`).
2. **O portão**, conferido antes de criar qualquer coisa: quem pede é Maestro, ou o usuário ligou **Automatic teams**; quem pede não é membro de outra equipe (sem equipes de equipes); há um `-reason`; no máximo 5 membros.
3. O formato `solo` só grava a decisão numa nota e **não cria terminal**.
4. Os outros criam os terminais (agente, **modelo**, papel), as arestas do formato, uma nota de **tarefa com o critério de aceite** e uma nota de **porquê**. Cada terminal nasce por `pty_spawn`.



Teste de ponta a ponta no app real: `dev-reviewer` criou Dev e Reviewer com o modelo `sonnet`, ligados como você ↔ Dev ↔ Reviewer.

Formatos: `solo`, `dev-reviewer`, `dev-qa`, `readers`, `parallel-devs`, cada um com "quando compensa" e o custo medido.

Como se usa

Como o NarraTer monta uma orquestração hoje

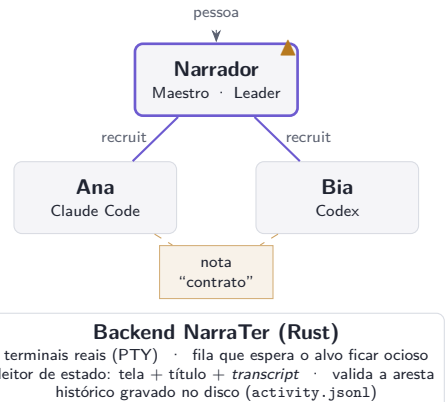
1. **A pessoa escolhe quem orquestra:** liga o **modo Maestro** num agente, ou a chave **Automatic teams**. O padrão continua sendo **um agente só** (D22).
2. **Se houver motivo, o líder monta a equipe**, um por um ou numa chamada (`team create`):

```
narrater recruit "Ana" --preset "Claude Code" --model sonnet
narrater recruit "Bia" --preset Codex --role Reviewer
```

   Cada um vira um terminal real, já ligado ao líder: **a aresta é a permissão** de conversar.
3. **Delega em paralelo.**

```
narrater ask --batch '{"Ana":"...","Bia":"..."}' --no-wait
narrater wait
```

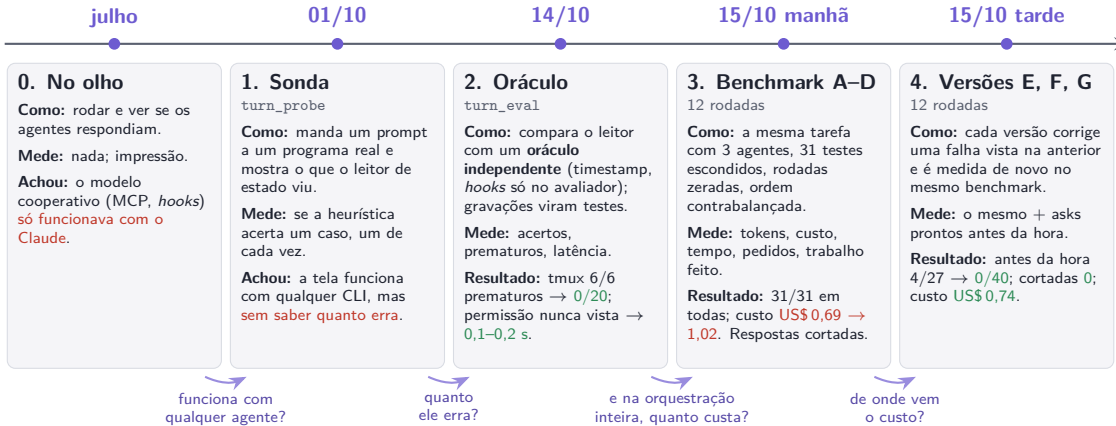
   O pedido é **digitado** no terminal do alvo quando ele fica ocioso.
4. **Recebe as respostas:** o NarraTer percebe o fim do turno, roda o aceite (`-check`) e devolve a resposta final.



Instruções chegam por arquivo, que qualquer CLI lê: *skills* e papéis como `CLAUDE.md`/`AGENTS.md`. Nenhum agente precisa de plugin ou protocolo próprio.

Como foi testado

Como os testes evoluíram



Cada etapa nasceu de uma pergunta que a anterior não respondia. O resultado da etapa 4 (nota máxima em tudo, só o custo variando) levou à pergunta seguinte: **um agente sozinho não faria igual?**

Como os testes evoluíram: equipe contra agente sozinho

16/10 — 16/10 — 16/10

5. Braço "sozinho"

tarefa `pedidos`, 3 rodadas

Como: o mesmo pedido a um Opus sozinho, contra as equipes F e G.

Mede: o grupo de controle que faltava.

Resultado: 31/31 também; 1,6 contra 3,9 min, 0,36 contra 2,50 milhões de tokens, US$ 0,55 contra 0,74.

6. Tarefa `armazem`

9 rodadas, 3 montagens

Como: 4 módulos independentes, feita para a equipe ganhar; **previsões gravadas antes** de rodar.

Mede: sozinho $\times$ equipe paralela $\times$ dev + revisor.

Resultado: 46/46 em todas; sozinho 1,8 min, equipe 2,8, revisor 2,3. A previsão "a equipe ganha" foi refutada.

7. Ponta a ponta

`bench/e2e/gerador-equipes.sh`

Como: o app real (build de release, tela virtual) com um `claude` falso, sem gastar cota.

Mede: se o gerador, o portão e o `-check` fazem o que dizem.

Resultado: sem `-reason`, recusado; `solo` não cria terminal; `-check false` volta `failed`.

Próximo: uma tarefa em que o agente sozinho erre ($< 70\%$), "Sonnet + revisor Opus" contra o Opus sozinho, e a montagem **gerada**: com Automatic teams ligado, o líder escolhe "solo" quando deve?